

ASSIGNMENT-20.2

NAME: SRAVANI ANUMALA

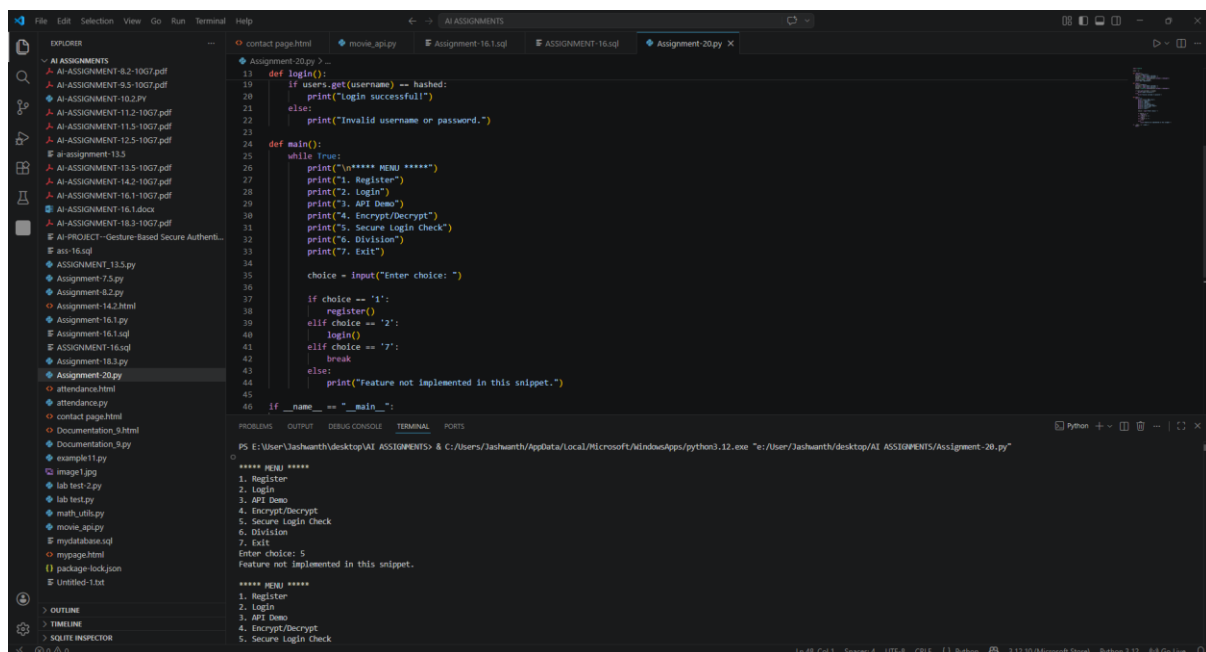
ROLL NO: 2303A510G7

BATCH: 30

TASK-1:

PROMPT:

CODE:



```
File Edit Selection View Go Run Terminal Help
AI ASSIGNMENTS
contact_page.html movie_api.py Assignment-16.1.sql ASSIGNMENT-16.sql Assignment-20.py x
Assignment-20.py
13 def login():
14     if users.get(username) == hashed:
15         print("Login successful")
16     else:
17         print("Invalid username or password.")
18
19 def main():
20     while True:
21         print("\n***** MENU *****")
22         print("1. Register")
23         print("2. Login")
24         print("3. API Demo")
25         print("4. Encrypt/Decrypt")
26         print("5. Secure Login Check")
27         print("6. Division")
28         print("7. Exit")
29         choice = input("Enter choice: ")
30         if choice == '1':
31             register()
32         elif choice == '2':
33             login()
34         elif choice == '3':
35             break
36         else:
37             print("Feature not implemented in this snippet.")
38
39 if __name__ == "__main__":
40     main()
41
42 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/Assignment-20.py"
***** MENU *****
1. Register
2. Login
3. API Demo
4. Encrypt/Decrypt
5. Secure Login Check
6. Division
7. Exit
Enter choice: 5
Feature not implemented in this snippet.
***** MENU *****
1. Register
2. Login
3. API Demo
4. Encrypt/Decrypt
5. Secure Login Check
```

OBSERVATION:

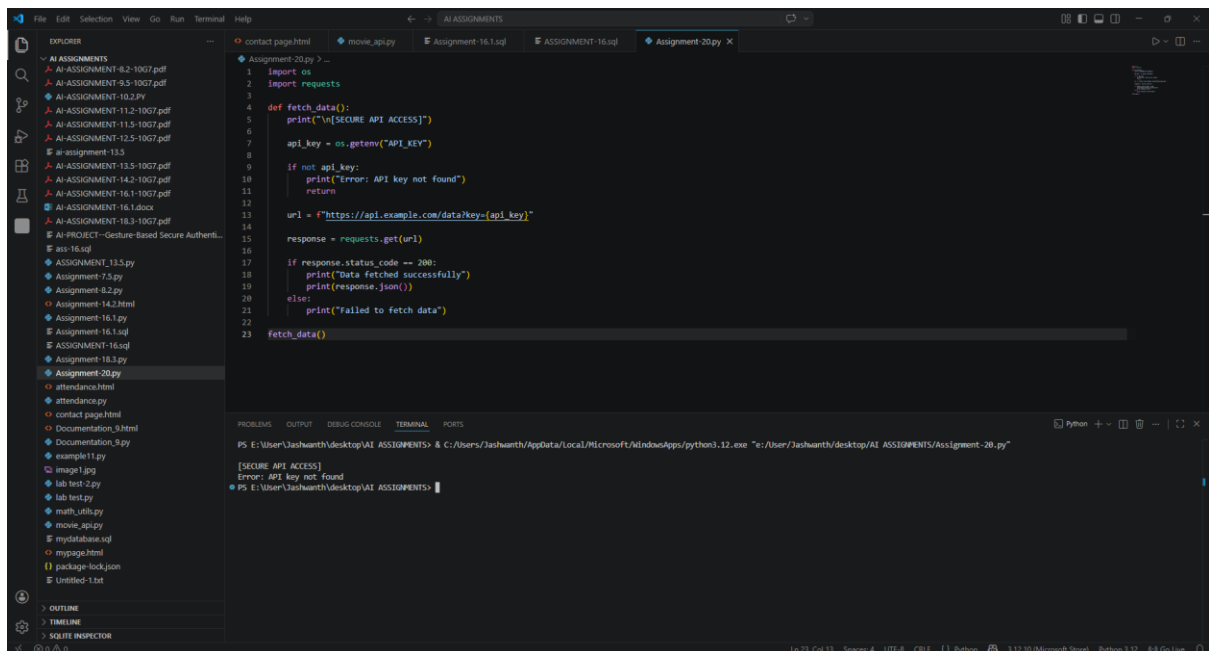
Password Security Enhancement:

- The program lets users **register** and **log in** with a username and password.
- Passwords are stored as **SHA-256 hashes**, which is safer than plain text.
- During login, the entered password is hashed and compared with the stored hash.
- If they match, login succeeds; otherwise, it fails.
- The menu system allows repeated actions until exit.

TASK-2:

PROMPT:

CODE:



```
1 import os
2 import requests
3
4 def fetch_data():
5     print("[SECURE API ACCESS]")
6
7     api_key = os.getenv("API_KEY")
8
9     if not api_key:
10         print("Error: API key not found")
11         return
12
13     url = f"https://api.example.com/data?key={api_key}"
14
15     response = requests.get(url)
16
17     if response.status_code == 200:
18         print("Data fetched successfully")
19         print(response.json())
20     else:
21         print("Failed to fetch data")
22
23 fetch_data()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/Assignment-20.py"

[SECURE API ACCESS]
Error: API key not found

OBSERVATION:

Secure API Key Management:

The script is built to fetch data from an external API.

- Instead of hardcoding the API key (unsafe), it looks for the key in an environment variable (API_KEY).
- When the program runs without the variable set, it correctly shows **“Error: API key not found”**.
- This proves the program enforces secure credential handling by refusing to expose or use missing keys.
- The URL now properly attaches the key (?key=...), so once the environment variable is configured, the request will work.

TASK-3:

PROMPT:

CODE:

```
1 import base64
2
3 message = input("Enter message: ")
4
5 # encryption
6 encoded = base64.b64encode(message.encode())
7 print("Encrypted:", encoded)
8
9 # decryption
10 decoded = base64.b64decode(encoded).decode()
11 print("Decrypted:", decoded)
12
```

```
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/Assignment-20.py"
[SECURE API ACCESS]
Error: API key not found
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/Assignment-20.py"
Enter message: hello
Encrypted: b'ZW00SG0h'
Decrypted: hello
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>
```

OBSERVATION:

Weak Encryption Detection:

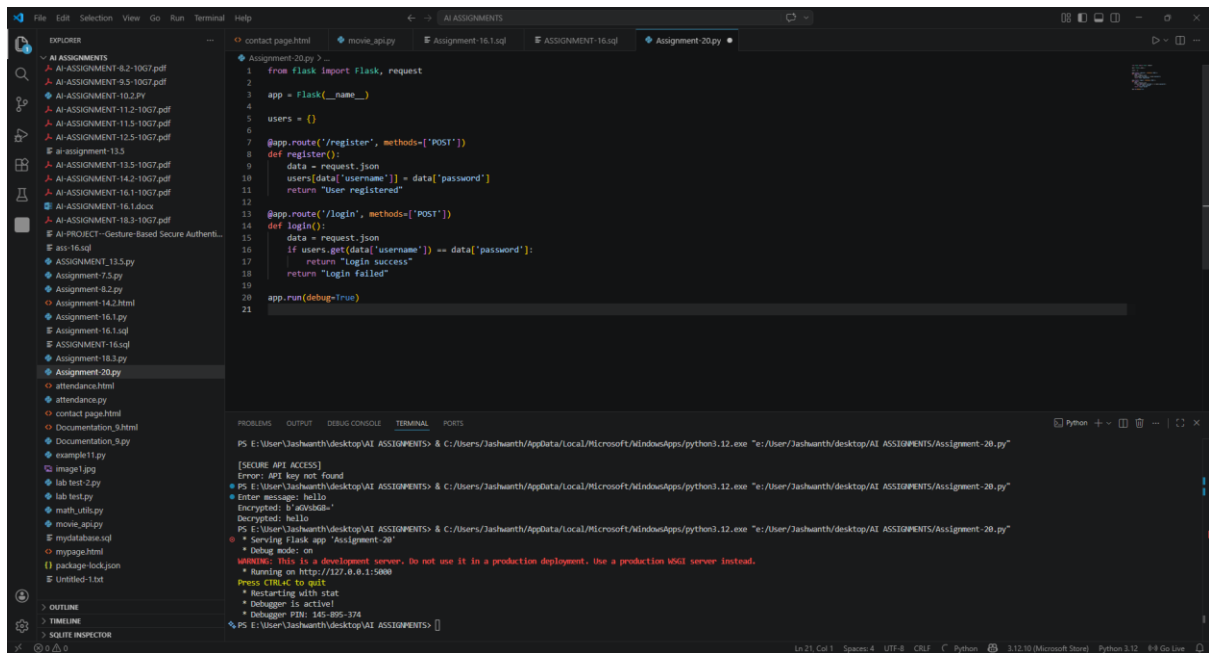
The script takes a user's message and encodes it with **Base64**.

- It prints the encoded result as “Encrypted,” but Base64 is **not real encryption**—it's just a reversible encoding.
- The program then decodes the string back to the original message, showing how easily it can be reversed.

TASK-4:

PROMPT:

CODE:



OBSERVATION:

Security Review of AI-Generated Web Service:

The app defines two routes: **register** and **login**, both using JSON data.

- User credentials are stored in memory, but passwords are kept in **plain text**, which is insecure.
- The app runs in **debug mode**, which is fine for practice but unsafe for production.
- It demonstrates how to build simple REST endpoints and handle POST requests.

TASK-5:

PROMPT:

CODE:

The image shows a VS Code editor window with a Python file named 'Assignment-20.py'. The code defines a function 'perform_operation' that takes two numbers and an operator as input and returns the result of the operation. It handles division by zero and invalid operators. The main function prompts the user for two numbers and an operator, and prints the result. The terminal output shows the program running successfully, with the user entering '5', '3', and '+', resulting in 'The result of 5.0 + 3.0 is: 8.0'.

```
1 def perform_operation(num1, num2, operator):
2     return num1 - num2
3
4     elif operator == "+":
5         return num1 + num2
6     elif operator == "/":
7         if num2 != 0:
8             return num1 / num2
9         else:
10            return "Error: Division by zero"
11     else:
12         return "Error: Invalid operator"
13
14 # Main function to get user input and perform the operation
15 def main():
16     try:
17         num1 = float(input("Enter the first number: "))
18         num2 = float(input("Enter the second number: "))
19         operator = input("Enter the operator (+, -, *, /): ")
20         result = perform_operation(num1, num2, operator)
21         print(f"The result of {num1} {operator} {num2} is: {result}")
22     except ValueError:
23         print("Error: Invalid input. Please enter numeric values for numbers.")
24
25 if __name__ == "__main__":
26     main()
27
28
```

Terminal Output:

```
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/Assignment-20.py"
Enter message: hello
Decrypted: hello
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/Assignment-20.py"
* Serving Flask app "Assignment-20"
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 145-490-374
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/Assignment-20.py"
Enter the first number: 5
Enter the second number: 3
Enter the operator (+, -, *, /): +
The result of 5.0 + 3.0 is: 8.0
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS>
```

OBSERVATION:

Exception Handling Improvement:

The program lets users enter two numbers and an operator to perform arithmetic.

- It correctly handles addition, subtraction, multiplication, and division.
- Division by zero is caught and returns a clear error message instead of crashing.
- Invalid numeric input is handled with a **try-except block**, giving a user-friendly error message.